

Break the Cycle

Manual de Ingeniería de Software

Runner 3D educativo contra bullying y ciberacoso

Documento	Tecnología
Manual técnico del proyecto	Unity 6 (6000.3.11f1)
Documentación de arquitectura y código	C# · URP
Uso interno / entrega académica	OpenXR (Nivel 3)

Equipo	Rol
Oscar Vélez	Gerente de proyectos
Luis Villarreal	Director de diseño
Franklin Oliviero	Director de pruebas
Joel Trespalacios	Director de UI/UX
Jesús de León	Director de documentación
Maikel Ortiz	Diseñador de niveles

Índice general

1	Introducción	3
1.1	Propósito del producto	3
1.2	Stack tecnológico	4
1.3	Convenciones	4
2	Visión arquitectónica	4
2.1	Capas del sistema	4
2.2	Los cinco TADs del dominio	5
2.3	Herencia y polimorfismo: coleccionables	5
2.4	Patrones de diseño aplicados	5
2.4.1	Observer (publicación-suscripción)	5
2.4.2	State (estados de partida)	5
2.4.3	Facade	6
2.4.4	Dependency Inversion (DIP)	6
3	Flujo de aplicación y escenas	6
3.1	Ruta del jugador	6
3.2	Catálogo de niveles	6
3.3	Desbloqueo y progreso	6
3.4	Sesión entre escenas	7
4	Autenticación y persistencia	7
4.1	Responsabilidad	7
4.2	Diagrama de clases (módulo Auth)	7
4.3	Flujo de registro e inicio de sesión	7
4.4	Formato en disco	8
4.5	Seguridad	8
5	Núcleo de gameplay: GameManager y sistemas	8
5.1	Composición del GameManager	8
5.2	Victoria, derrota y pausa	9
5.3	Leaderboard	9
5.4	Suscripción UI (Observer)	9
6	Nivel 1: Runner por carriles	9
6.1	Bootstrap y composición de escena	9
6.2	Jugador y spawner	9
6.3	Secuencia de inicio: cinemática → HUD	10
7	Nivel 2: Sandbox y decisiones morales	10
7.1	Mecánica	10
7.2	Decisiones en slow motion	10

7.3	Integración con dominio	10
8	Nivel 3: Realidad virtual y escritorio	11
8.1	Selección de modo de entrada	11
8.2	XRModeManager	11
8.3	Combate y oleadas	11
8.4	UI en VR	11
8.5	Persistencia de sesión de nivel	11
9	Interfaz de usuario y accesibilidad	12
9.1	Menús	12
9.2	Accesibilidad	12
10	Infraestructura transversal	12
11	Datos y configuración	12
11.1	Estructuras JSON principales	12
12	Ampliaciones y fallos habituales	12
12.1	Nuevo coleccionable	12
12.2	Nueva UI suscrita a eventos	13
12.3	Pantalla tras la cinemática	13
12.4	Errores frecuentes	13
A	Glosario	13
B	Escenas y scripts principales	13
C	Estructura de carpetas	14

Índice de figuras

2.1	Capas lógicas del proyecto	4
3.1	Flujo principal entre escenas	6
4.1	Diagrama UML del módulo de autenticación	7
6.1	Secuencia Nivel 1 tras la cinemática	10

Capítulo 1. Introducción

1.1. Propósito del producto

Break the Cycle es un videojuego educativo en tercera persona desarrollado por **UnderRise Studios**. El jugador avanza por entornos escolares hostiles representando la presión del acoso (presencial en el Nivel 1, ciberacoso y combate en el Nivel 3). La mecánica combina **runner por carriles**, **sandbox con decisiones morales** y **combate en realidad virtual o escritorio**, con narrativa de dificultad creciente entre niveles.

El manual está dirigido al equipo de desarrollo que mantiene escenas, mecánicas, UI y persistencia del proyecto.

1.2. Stack tecnológico

Tabla 1.1: Componentes del proyecto

Componente	Detalle
Motor	Unity 6000.3.11f1 (<code>ProjectSettings/ProjectVersion.txt</code>)
Lenguaje	C# (.NET según configuración del proyecto)
Render	URP (Universal Render Pipeline)
Entrada	Unity Input System (paquete en <code>Packages/manifest.json</code>)
XR (Nivel 3)	XR Plug-in Management + OpenXR ; XR Interaction Toolkit
Persistencia	JSON y JSONL en disco (<code>GameDataPaths.DataRoot</code>)
UI	uGUI + TextMeshPro donde aplica

1.3. Convenciones

- **Escenas** en Build Settings: `Auth`, `MainMenu`, `LevelMenu`, `Level_1`, `Level_2`, `Level_3` (constantes en `SceneNames`).
- **Identificadores de nivel**: enum `LevelId` (`Level1_Runner`, `Level2_SandboxRpg`, `Level3_FinalBoss`).
- Rutas de código: prefijo `Assets/...` respecto a la raíz del proyecto Unity.
- **Namespaces** de VR/combate: principalmente `MundoMental.VR`, `MundoMental.VR.Combat`, `MundoMental.VR.UI`.

Capítulo 2. Visión arquitectónica

2.1. Capas del sistema

El proyecto se divide en tres capas (ver fig. 2.1).

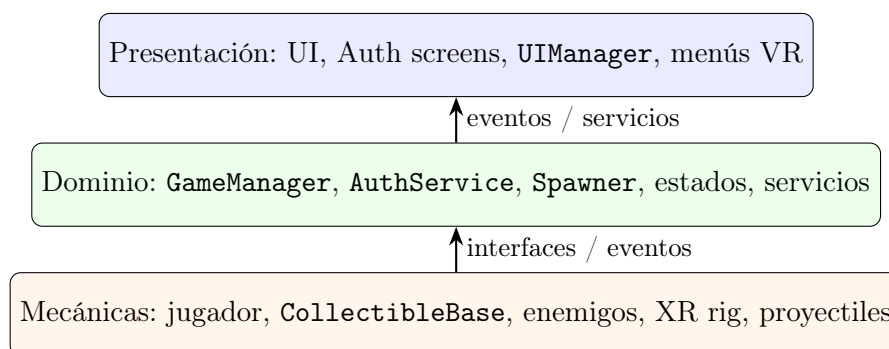


Figura 2.1: Capas lógicas del proyecto

La pantalla de login **no** calcula hashes: delega en `AuthService`. La barra de vida **no** lee variables privadas del jugador: se suscribe a eventos del dominio (patrón Observer).

2.2. Los cinco TADs del dominio

Un *Tipo Abstracto de Datos* aquí es una clase núcleo con responsabilidad única (SRP):

Clase y archivo	Responsabilidad
GameManager	
Assets/Core/Architecture/GameManager.cs	Orquesta vida, puntaje, meta, victoria y game over; fachada sobre HealthSystem, ScoreSystem y GameStateManager.
AuthService	
Assets/Core/Auth/AuthService.cs	Registro e inicio de sesión; validación; sin dependencia de UI.
Spawner	
Assets/Core/Mechanics/Spawner.cs	Filas de obstáculos y coleccionables en tres carriles (Nivel 1).
RisingLavaController	
Assets/Core/Mechanics/RisingLavaController.cs	Lava ascendente y mensajes narrativos.
SlowMotionChoiceTrigger	
Assets/Core/Mechanics/SlowMotionChoiceTrigger.cs	Decisiones en cámara lenta (ignorar / confrontar).

2.3. Herencia y polimorfismo: coleccionables

CollectibleBase (Assets/Core/Mechanics/CollectibleBase.cs) es una clase **abstracta**. Define animación, trigger de recogida y destrucción; cada hijo implementa ApplyPickup(GameManager).

Hijos: HealthCollectible, ScoreCollectible, SpeedBoostCollectible. El Spawner instancia prefabs que son CollectibleBase; no usa if (tipo == corazón).

OCP (Open/Closed): un nuevo ítem (p. ej. escudo) se añade como subclase + prefab sin modificar CollectibleBase ni Spawner.

2.4. Patrones de diseño aplicados

2.4.1. Observer (publicación–suscripción)

- **Sujetos** (notifican): HealthSystem.OnHealthChanged, ScoreSystem.OnScoreChanged, GameStateManager (OnGameOver, OnLevelComplete, etc.).
- **Fachada de eventos:** GameManager reexpone los eventos hacia la UI.
- **Observadores:** UIManager (HUD Nivel 1/2), PlayerHealthHud (VR), paneles de victoria/derrota.

La UI se entera de los cambios por eventos; no hace falta consultar la vida en cada Update.

2.4.2. State (estados de partida)

GameStateManager (Assets/Core/Architecture/GameStateManager.cs) usa BaseGameState y estados concretos: PlayingState, PausedState, GameOverState, VictoryState. Al pausar,

`PausedState.Enter()` fija `Time.timeScale = 0`; no se reparten if (pausado) por todo el código.

2.4.3. Facade

`GameManager` agrupa tres subsistemas inicializados por separado; la UI y el jugador tienen un único punto de acceso (`CurrentHealth`, `TakeDamage`, eventos).

2.4.4. Dependency Inversion (DIP)

`IPlayerController` (`Assets/Core/Player/IPlayerController.cs`) define `DistanceTraveled`, `PlayerTransform` y `Kill`. Implementaciones: `PlayerController` (Nivel 1), `SandboxPlayerController` (Nivel 2). Obstáculos y coleccionables dependen del **contrato**, no del tipo concreto.

Capítulo 3. Flujo de aplicación y escenas

3.1. Ruta del jugador

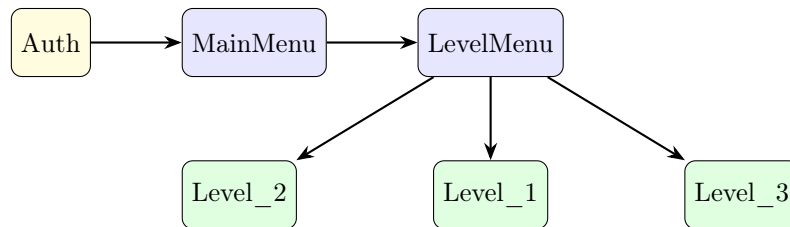


Figura 3.1: Flujo principal entre escenas

3.2. Catálogo de niveles

`LevelsCatalog` (`Assets/Core/Infrastructure/Progress/LevelsCatalog.cs`) resuelve nombre de escena y etiqueta visible:

```
1 public static string GetSceneName(LevelId id) => id switch
2 {
3     LevelId.Level1_Runner => SceneNames.Level_1,
4     LevelId.Level2_SandboxRpg => SceneNames.Level_2,
5     LevelId.Level3_FinalBoss => SceneNames.Level_3,
6     _ => SceneNames.Level_1
7 };
```

Listing 3.1: Mapeo `LevelId` → escena

3.3. Desbloqueo y progreso

`GameProgressService` persiste en el perfil del usuario (`PlayerProfileData`): niveles completados, récords, snapshot de pausa. `LevelMenuManager` consulta `GameProgressService.IsLevelUnlocked` antes de cargar `Level_3`.

3.4. Sesión entre escenas

- AccountSession: usuario actual en memoria durante la ejecución.
- SessionPersistence: archivo session.json en GameDataPaths (no PlayerPrefs en la versión actual).
- Al volver a Auth, MainMenu o LevelMenu, LevelStartPersistence.ClearAll() limpia flags de cinemática/instrucciones de la sesión de juego (Nivel 3).

Capítulo 4. Autenticación y persistencia

4.1. Responsabilidad

Registro e inicio de sesión locales (sin servidor), progreso y leaderboard por usuario, contraseñas con hash y sal.

4.2. Diagrama de clases (módulo Auth)

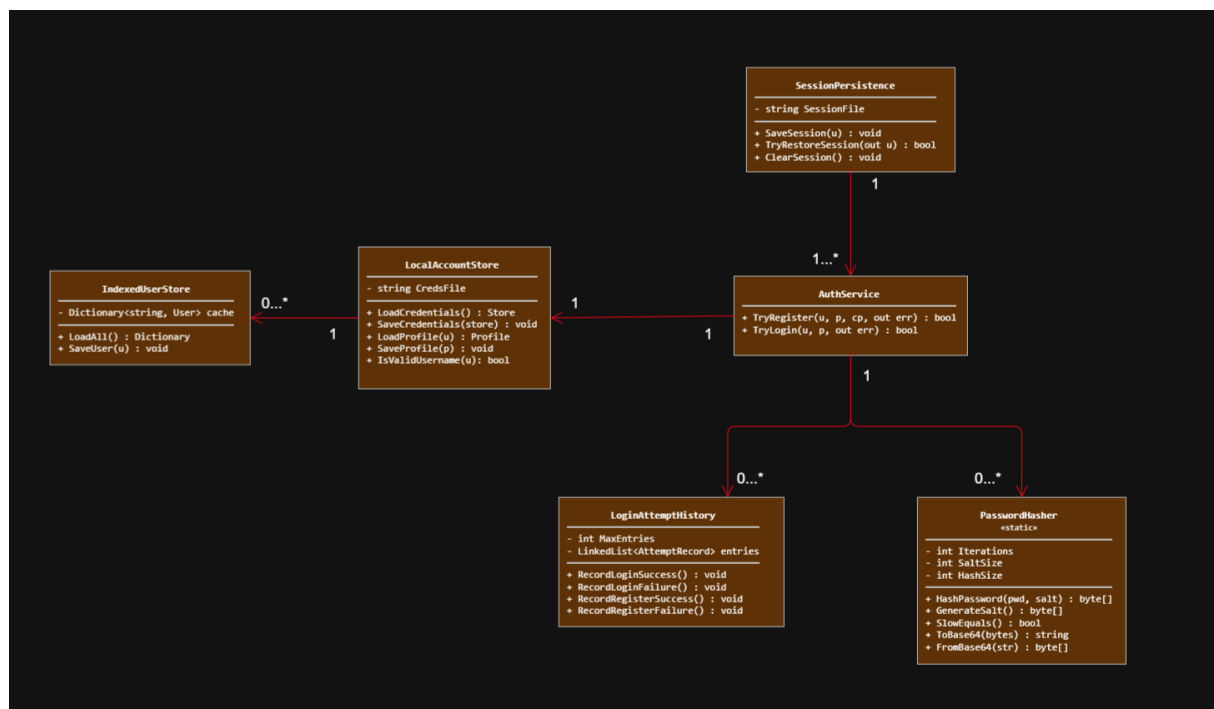


Figura 4.1: Diagrama UML del módulo de autenticación

4.3. Flujo de registro e inicio de sesión

Registro (`AuthService.TryRegister`):

1. Validar usuario (`LocalAccountStore.IsValidUsername`: 3-24 caracteres, alfanumérico y `_`).
2. Validar contraseña (mínimo 4 caracteres, coincidencia con confirmación).
3. Comprobar que el usuario no exista (`LoadCredentials`).

4. Generar sal y hash (`PasswordHasher`).
5. Persistir en `IndexedUserStore.WriteAllUsers`.
6. Crear perfil por defecto y guardar sesión.

Login (`AuthService.TryLogin`): carga credenciales, localiza usuario, verifica hash con `PasswordHasher.SlowlyVerify`, restaura `AccountSession` y `SessionPersistence.SaveSession`.

4.4. Formato en disco

Tabla 4.1: Archivos bajo `GameDataPaths.DataRoot`

Archivo / carpeta	Contenido
<code>users.data</code>	Una línea JSON por usuario (<code>UserCredentialEntry</code>)
<code>users.index</code>	Índice usuario númeroDeLínea para búsqueda directa
<code>Profiles/profile_*.json</code>	<code>PlayerProfileData</code> (récores, niveles completados)
<code>session.json</code>	Sesión activa (<code>SessionFileData</code>)
<code>Settings/</code>	Ajustes de accesibilidad y juego

En **Editor**, `DataRoot` apunta a `<raíz proyecto>/Data/`; en **build**, a `Application.persistentDataPath/Br`

4.5. Seguridad

- No se almacena la contraseña en texto plano.
- Se guardan `saltBase64` y `hashBase64` por usuario.
- `LoginAttemptHistory` registra fallos para auditoría local.
- `IndexedUserStore.MigrateFromLegacyJsonIfPresent` migra `credentials.json` antiguo sin bloquear el arranque si falla.

```

1  // Assets/Core/Architecture/HealthSystem.cs
2  public void ApplyHealthDelta(float delta)
3  {
4      currentHealth = Mathf.Clamp(currentHealth + delta, 0f, maxHealth
5      );
6      OnHealthChanged?.Invoke(currentHealth);
7      if (currentHealth <= 0f) OnDeath?.Invoke();
8  }
```

Listing 4.1: Cambio de vida (`HealthSystem`)

Capítulo 5. Núcleo de gameplay: `GameManager` y sistemas

5.1. Composición del `GameManager`

`GameManager` no concentra toda la lógica: **compone** tres componentes en el mismo `GameObject` (o referenciados):

Sistema	Rol
HealthSystem	Vida máxima, daño, curación, evento <code>OnDeath</code>
ScoreSystem	Puntuación y <code>OnScoreChanged</code>
GameStateManager	Estados Playing / Paused / GameOver / Victory

Inicialización en `Start: healthSystem.Initialize(), scoreSystem.Initialize(), stateManager.Initialize()`. El nivel activo se detecta por nombre de escena (`DetectCurrentLevel`).

5.2. Victoria, derrota y pausa

- **Victoria:** meta de distancia (`GoalDistance`) o arco de meta (`FinishArchTrigger`); `HandleLevelComplete` llama a `GameProgressService.RecordLevelComplete`.
- **Game over:** vida ≤ 0 ; `StartGameOverSequence` con retardo configurable; razones `Generic`, `Caught`, `Lava`.
- **Pausa:** `PauseMenuController`; snapshot vía `GameProgressService.SavePauseSnapshot` si hay sesión.

5.3. Leaderboard

`LeaderboardService` (`Assets/Core/Auth/LeaderboardService.cs`) construye textos Top-3 global por nivel para el menú de puntuaciones (`MenuManager.OnScoresClicked`).

5.4. Suscripción UI (Observer)

```

1 // Assets/Core/UI/UIManager.cs      Start()
2 gm.OnHealthChanged += OnHealthChanged;
3 gm.OnScoreChanged += OnScoreChanged;
4 gm.OnGameOverWithReason += ShowGameOverWithReason;
5 gm.OnLevelComplete += ShowVictory;
6 // OnDestroy: desuscripción con -=

```

Listing 5.1: Observador: UIManager

Capítulo 6. Nivel 1: Runner por carriles

6.1. Bootstrap y composición de escena

`Level1Bootstrap` (`Assets/Core/Levels/Level1Bootstrap.cs`, orden de ejecución -200) configura música, resuelve al jugador y ajusta `GoalDistance` según la posición del `FinishArch` en escena.

La escena `Level_1` incluye prefabs de pasillo, `GameRoot` (contenedor de gameplay desactivado al inicio), cinemática y `GameUI`.

6.2. Jugador y spawner

- **PlayerController:** tres carriles, salto, avance automático, `IPlayerController`.
- **Spawner:** filas de obstáculos y coleccionables; evita solapamientos en carriles.

- `ChasePursuer` + `PursuitStressController` + `CameraFollow`: presión visual según vida.
- `RisingLavaController`: amenaza ascendente y mensajes narrativos.

6.3. Secuencia de inicio: cinemática → HUD

`GameUI` es hijo de `GameRoot` y puede quedar con `activeSelf = false`. Activar solo el padre no enciende esos hijos; entonces `UIManager.Start()` no se ejecuta y el HUD no aparece.

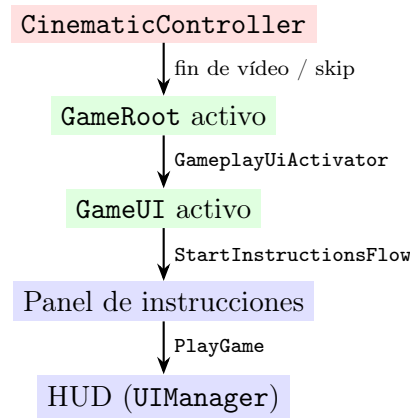


Figura 6.1: Secuencia Nivel 1 tras la cinemática

Clases involucradas:

- `CinematicController` (`Assets/Scripts/MundoMental/UI/CinematicController.cs`): desactiva `GameRoot` al inicio; al terminar activa `gameplay` y dispara `OnCinematicFinished`.
- `GameplayUiActivator`: busca `GameUI` bajo `GameRoot` y lo activa.
- `InstructionsUIController`: panel de instrucciones; `timeScale = 0` hasta `PlayGame`.

Capítulo 7. Nivel 2: Sandbox y decisiones morales

7.1. Mecánica

`SandboxPlayerController` implementa `IPlayerController` con movimiento libre en mapa (no carriles). `Level2Bootstrap` prepara la escena. La meta es **distancia recorrida** hacia un objetivo lejano; `Level2HUDBars` muestra barras específicas.

7.2. Decisiones en slow motion

`SlowMotionChoiceTrigger` presenta UI de dilema (ignorar / confrontar), reduce `timeScale` y aplica consecuencias narrativas o de `gameplay` según configuración en Inspector.

7.3. Integración con dominio

El mismo `GameManager` detecta `Level2_SandboxRpg` por escena; al completar, `GameProgressService.Record` guarda tiempo transcurrido.

Capítulo 8. Nivel 3: Realidad virtual y escritorio

8.1. Selección de modo de entrada

`Level3InputSelectorUI` pausa el juego (`timeScale = 0`), muestra selector VR / Escritorio y llama a `XRModeManager`:

- **VR:** `ActivateVRMode` — carga `OpenXR`, activa `XR Origin`, desactiva `DesktopPlayer`.
- **Escritorio:** `ActivateDesktopMode` — `DesktopLevel3Player` (FPS con ratón).

Al completar la selección, evento `OnSelectionComplete` → `InstructionsUIController.StartInstruction`

8.2. XRModeManager

Archivo: `Assets/Scripts/MundoMental/XRModeManager.cs`. Responsabilidades:

1. Inicializar loader `OpenXR` con reintentos y tiempo de gracia (`XRSettings.isDeviceActive`).
2. Intercambiar rigs y cámaras (`ConfigureVrPresentation`, `DisableDesktopPresentation`).
3. Plantillas visuales para cuerpo en desktop (`DesktopPlayerBodyVisuals`).
4. Fallback a escritorio si `OpenXR` falla en build de jugador.

8.3. Combate y oleadas

Namespace `MundoMental.VR.Combat`: `VRHandPower`, `ShieldController`, `HandEquipmentTracker`, armas (`SwordWeapon`, `GunWeapon`), `PlayerHealth`.

`WaveSpawner` (`Assets/Scripts/MundoMental/Environment/WaveSpawner.cs`) espera instrucciones si `InstructionsUIController` está activo; luego inicia oleadas. `BossFightManager` gestiona el jefe final.

8.4. UI en VR

- `VrPresentationUi`: coloca canvas `World Space` delante de la cámara del HMD.
- `InstructionsUIController`: paneles VR vs desktop según rig activo.
- `PlayerHealthHud`: HUD de vida en mano o mundo; escucha `PlayerHealth`.
- `GameOverMenu` / `AllSkeletonsClearedMenu`: paneles world-space con `XRUIInputModule`.

8.5. Persistencia de sesión de nivel

`LevelStartPersistence` (en `XRModeManager.cs`): evita repetir cinemática/instrucciones al reiniciar el mismo nivel en la misma ejecución; se limpia al volver a menús principales.

Capítulo 9. Interfaz de usuario y accesibilidad

9.1. Menús

Componente	Rol
AuthMenuController / AuthUiFactory	Login y registro
MenuManager	Menú principal: jugar, créditos, puntuaciones, logout
LevelMenuManager	Selección de nivel y bloqueos
SettingsMenuController	Sensibilidad, volumen, daltonismo
PauseMenuController	Pausa in-game

9.2. Accesibilidad

GameSettings (Assets/Core/Settings/GameSettings.cs) persiste opciones; evento estático GameSettings.Changed. ColorblindCameraFilter, subtítulos y señas vía AccessibilitySettingsTarget en el canvas de juego.

Capítulo 10. Infraestructura transversal

- MusicPlayer: música en bucle por escena (Level1Bootstrap usa chasingsound).
- AtomicFile.WriteAllText: escritura atómica de JSON para evitar corrupción.
- GameSettingsBootstrap / GameSettingsSceneApplier: aplicar ajustes al cargar escena.
- AgentDebugger: trazas de depuración XR (desarrollo).

Capítulo 11. Datos y configuración

11.1. Estructuras JSON principales

- UserCredentialEntry: usuario, saltBase64, hashBase64.
- AuthCredentialsStore: lista de usuarios (carga completa vía ReadAllUsers).
- PlayerProfileData: récords por nivel, flags levelXCompleted, lastPauseRun.
- SessionFileData: active, username.

Capítulo 12. Ampliaciones y fallos habituales

12.1. Nuevo coleccionable

1. Crear clase que herede de CollectibleBase.
2. Implementar ApplyPickup(GameManager gm).
3. Crear prefab con collider trigger en el root.
4. Añadir prefab a la lista del Spawner (sin modificar código del spawner).

12.2. Nueva UI suscrita a eventos

1. Suscribirse en `OnEnable` o `Start` a eventos de `GameManager`.
2. Desuscribirse en `OnDestroy`.
3. No leer vida en `Update` si existe evento equivalente.

12.3. Pantalla tras la cinemática

1. Encadenar `UnityEvent` en `CinematicController.OnCinematicFinished`.
2. Asegurar que `GameUI` (o el canvas) esté activo — usar `GameplayUiActivator`.
3. No asumir que `SetActive(true)` en el padre activa hijos con `activeSelf` false.

12.4. Errores frecuentes

Síntoma	Causa probable
HUD no aparece tras cinemática	<code>GameUI</code> hijo inactivo bajo <code>GameRoot</code>
Panel VR invisible	Canvas <code>World Space</code> con cámara del <code>DesktopPlayer</code> desactivado
OpenXR no inicia con selector pausado	<code>timeScale = 0</code> ; restaurar antes de <code>ActivateVRMode</code>
Instrucciones no bloquean oleadas	<code>WaveSpawner</code> no espera <code>OnGameplayStarted</code>

Apéndice A. Glosario

TAD Tipo Abstracto de Datos; clase núcleo con responsabilidad única en el dominio.

Observer Patrón donde sujetos notifican cambios y observadores reaccionan sin acoplamiento directo.

URP Universal Render Pipeline de Unity.

OpenXR API estándar para dispositivos VR; usada en Nivel 3 vía XR Plug-in Management.

JSONL Formato de una línea JSON por registro (`users.data`).

Apéndice B. Escenas y scripts principales

Escena	Scripts destacados
Auth	<code>AuthMenuController</code> , <code>AuthSceneBootstrap</code>
MainMenu	<code>MenuManager</code> , <code>MainMenuBootstrap</code>
LevelMenu	<code>LevelMenuManager</code>
Level_1	<code>Level1Bootstrap</code> , <code>CinematicController</code> , <code>GameManager</code> , <code>PlayerController</code>
Level_2	<code>Level2Bootstrap</code> , <code>SandboxPlayerController</code> , <code>SlowMotionChoiceTrigger</code>
Level_3	<code>XRModeManager</code> , <code>Level3InputSelectorUI</code> , <code>WaveSpawner</code> , <code>DesktopLevel3Player</code>

Apéndice C. Estructura de carpetas

Assets/Core/

Auth/	AuthService, LocalAccountStore, LeaderboardService
Architecture/	GameManager, HealthSystem, ScoreSystem, GameStateManager
Mechanics/	Spawner, CollectibleBase, RisingLavaController, ...
UI/	UIManager, MenuManager, PauseMenuController, ...
Settings/	GameSettings, accesibilidad
Levels/	Level1Bootstrap, Level2Bootstrap, ...
Infrastructure/	GameDataPaths, IndexedUserStore, MusicPlayer, Progress/

Assets/Scripts/MundoMental/

VR/	XRModeManager, VRFirstPersonFullBody, ...
VR/Combat/	Armas, escudo, vida, menús derrota/victoria VR
VR/UI/	CinematicController, InstructionsUIController, ...
VR/Environment/	WaveSpawner, BossFightManager